1. In a small school, the teachers are adopting a new grading system to evaluate students' performance. Each student receives a score, and the school wants to convert these scores into grades. The grading system is as follows:

   A: 90-100 B: 75-89 C: 60-74 D: 45-59 F: Below 45

   Write a Java program to help the school convert the scores into grades.

2. Imagine you are going on a magical week-long weather adventure in a land known for its diverse climates. Each day, you need to record the temperature of the region you are exploring. To make sense of your weather data, you need to create a Java to store these temperatures in an array and find the highest, lowest, and average temperature.

3. Develop a Java program to demonstrate method overloading using a Shape class. Create overloaded area() methods to calculate the area of a circle, rectangle, and square using different parameters.

4. Develop a Java program to demonstrate static variables and static methods in a Student Management System. Create a Student class with a static variable to count the number of students and a static method to display the total number of students.

5. Implement classes and object by creating a Java class named Employee that contains the employee's **name** and **salary**. Initialize these values using a constructor. Define a method calculateBonus() to calculate and return the employee's bonus as **10% of the salary**. In the main() method, create an Employee object and display the employee's name, salary, and calculated bonus.

6. Implement types of constructors by developing a **banking application** where each customer can have multiple bank accounts. Create two separate BankAccount objects for Alice: one with a **default balance of 0** and another with a **user-specified balance**. Use the getBalance() and setBalance() methods to initialize and display the balances.

7. Develop a Java program to demonstrate constructor overloading using a Box class with default, single-parameter, and three-parameter constructors, and calculate the volume of the box.

8. Implement data encapsulation by developing a Payroll System organize employee and salary information. Create an Employee class in the employeedetails package to store employee details and a Salary class in the salarydetails package to calculate gross and net salary. Use the provided PayrollMain class to generate and display the employee's pay slip.

   Requirements:

   Employee should store empId, name, department, and basicPay using private variables, a constructor, and getter methods.

   Salary should calculate:

   Gross Salary = Basic Pay + 80% DA + 20% HRA

   Net Salary = Gross Salary − 10% Tax

   generatePaySlip(Employee emp) should calculate and display the employee's pay-slip details.

9. Develop a Java program for a Student Management System using single inheritance. Create a Student class to store student ID and marks in Java, C, and C++. Derive a Result class to calculate and display the total and average marks.

10. Develop a Java program to implement multilevel inheritance using the hierarchy Person → Doctor → Surgeon. Demonstrate constructor execution at each level and display the complete details of a surgeon.

11. Develop a Java program to demonstrate **multiple inheritance using interfaces** in a University Management System. Create interfaces Student and Sports, and implement both interfaces in a StudentProfile class to display student academic and sports details.

12. Develop a Java program to demonstrate hierarchical inheritance in an Employee Management System. Create a superclass Employee and derive Manager, Developer, and Tester classes from it to display the details and roles of different employees.

13. Develop a Java program to demonstrate method overriding using inheritance in a Product Inventory System, where ElectronicProduct extends Product and overrides the displayInfo() method to display product details along with the warranty period.

14. A geometry application needs to calculate the areas of different shapes using abstract classes. Develop a Java program with an abstract class CalcArea that declares methods to calculate the areas of a triangle, rectangle, square, and circle. Create a class FindArea that extends CalcArea and implements all the area methods.

15. Develop a Java program to demonstrate **loose coupling** in an Online Shopping Payment System using an interface PaymentMethod. Implement the interface using **CreditCardPayment** and **UPIPayment** classes. Use an interface reference to process payments and display the payment receipt for the selected payment method.

16. Develop a Java program using Java 8+ interface features for a Delivery Management System. Create an interface DeliveryService with a default method calculateDeliveryCharge() and a static method validateShipment(). Implement the interface in a CourierDelivery class and display the delivery details and charges.